

Day 10 — Content Modifier (25 Aug 2026)

It is located under the Transformation Palette functions.

Message Transformation	→	Connectivity
	→	Message Transformation

So here, by using all these Palette functions, we will do the second process (Message Transformation) — recapping the two-step pattern from Day 9 (Connectivity, then Message Transformation).

Content Modifier → **Incoming message** → **modified**. It's used to **add additional data to the incoming payload**, with three possible actions: **Add, Modify, Delete**.

Data will be added at:

- Message Header
- Message Body
- Message Property (*also called **Exchange Property***)

Both share the same structural fields when you create an entry:

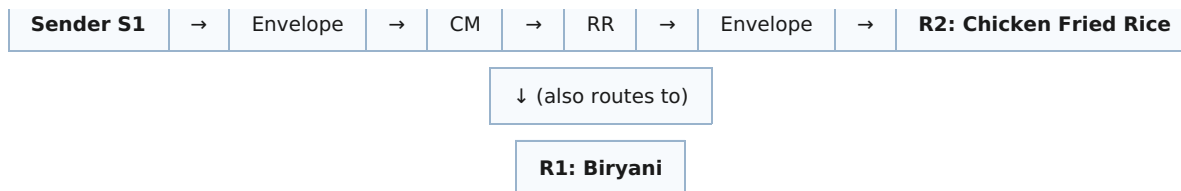
Field
Action
Name
Source Type
Source Value
Data Type

Message Header vs. Exchange Property — structurally, no difference in terms of the fields you fill out.

	What Happens
Message Header	Whatever variables are created in the Message Header will reach the Target System (Receiver)
Exchange Property	Whatever variables are created in the Exchange Property will be used only within the Integration Flow — they never leave CPI

This is the single most important distinction in all of CPI message handling — and it's exactly the Header vs. Property distinction we covered back when debugging the "Content Modifier not sending the token" issue earlier. If a value needs to travel to the receiver (like an Authorization token), it must be a **Header**. If it's only needed internally for logic/routing within the same flow, it should be a **Property** — putting it in the wrong place is the most common real-world mistake.

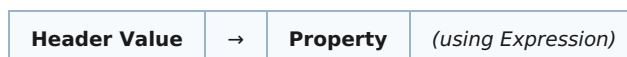
Scenario: Sender (**Zomato**) → Delivery Agent (**Satish**).



Content Modifier Headers Scenario

What this illustrates: Zomato's "Hot Box" contains two items — Biryani and Chicken Fried Rice. The delivery agent (Satish) needs to know which specific item goes to which specific destination. **Content Modifier headers** are what carry this routing/identification information along with the message — e.g., a header identifying "this package = Biryani" so the receiving step (or system) can correctly route **R1 → Biryani** and **R2 → Chicken Fried Rice** separately, even though both started from the same single "Hot Box" sender.

6. The Core Concept — Header Value to Property, Using Expression



Temporarily — this transfer/reference is a temporary, in-flow mechanism.

7. Source Types — Full List (Verified)

By using Source Type, we decide **HOW** we will pass the data.

#	Source Type	What It Does
1	Constant	A fixed, custom value created by the user
2	Expression	Dynamic value using Camel Simple Expression syntax
3	Global Variable	A variable with tenant-wide scope, usable across different iFlows
4	Header	Pulls the value from an already-existing header
5	Local Variable	A variable scoped only to the current integration flow
6	Number Range	Auto-generates a unique sequence number
7	Property	Pulls the value from an already-existing property
8	XPath	Extracts a specific value directly from the XML message body

Verified accurate against SAP's official documentation — all 8 source types and their purposes match exactly.

Constant — Worked Example from Class

Action	Name	Source Type	Source Value
Create	Name	Constant	Satish

8. Demo: Trace, End to End

Trace: end to end, to process flow.

Postman Demo Steps

1. Postman → Deployed URL → **POST**
2. Body → **raw** → **XML**

```
<orders>
  <od-name>Iron</od-name>
  <od-price>234</od-price>
</orders>
```

S	→	Integration Process (Envelope → CM) → Envelope	→	R
---	---	--	---	---

Content Modifier — Create a Header

Create	Name	ST (Source Type)	SV (Source Value)	D-Type
Header	Name	Constant	Bharath	String

Result: In Postman, after making the above change and sending the request again, we will get the additional header called: **Name: Bharath** — confirming the header was successfully created and passed through to the response.

9. Second Scenario — Using Message Body with Expression

Second scenario: using → Message Body → `${in.body}` (Expression)

How CPI's Expression Syntax Works

CPI → works on the Apache [Camel] Framework → expressions will be called with `${ }`

Common Expression Patterns

Suppose you want to call a field/variable that was created in the header, into the body — then:

Expression	Purpose
<code>\${header.Name}</code>	Calls a header named "Name"
<code>\${in.body}</code>	Calls the current message body
<code>\${header.headername}</code>	Generic pattern for calling any header by name
<code>\${property.propertyname}</code>	Generic pattern for calling any property by name

10. Property — Worked Example

Create	Name	Source Type	Source Value
Property	Order_Price	Constant	12345

Important Test Question From Class

Question: Now, will this property be visible in Postman? (Yes or No)

Answer: No — it is restricted only to the iFlow itself.

This is a direct, practical confirmation of the Header vs. Property rule from Section 4 above — a Property, by design, never shows up externally (e.g., in the Postman response), no matter how you create it.

11. How to Use an Exchange Property's Value Inside a Header

If we want to use any Exchange Property's name/value inside a Header, then what do we do?

Answer: We need to create **another Content Modifier**. In that new Content Modifier, we create a **new variable under Header (Message)**, where the **Source** will be the **previously created Exchange Property's name** (i.e., Source Type = Property, Source Value = the property's name).

Message Header — Full Field Example

Action	Name	Source Type	Source Value	Data Type
Create	?	Expression	<code>\${property.<propertyname>}</code>	java.lang.String

12. Requirement — The Three-Layer Flow (Recap)





(*Request Reply*) — this is the general pattern whenever the middle system (CPI) needs to actively fetch/exchange data as part of the flow, as covered in Day 9's External Call section.

13. Custom Headers — Allowing Them Through

Explained about: Custom headers allowing (*) → Username → Allowed Headers.

Context, tying back to earlier debugging: This connects directly to the Postman header-capturing troubleshooting we worked through earlier in this project — CPI's sender adapter needs custom headers to be explicitly allowed (sometimes via a wildcard * or an explicit allow-list) before they'll be captured and usable inside the iFlow. This is exactly the kind of configuration check that resolves "why isn't my custom header showing up" issues.